INTERVIEW ASSIGNMENT

Agentic SDLC Orchestrator for a URL Shortener

A requirement goes in; a governed, reviewable engineering outcome comes out - agents execute under explicit autonomy boundaries, humans own approvals and final quality.

Stack	Java 21, Spring Boot 3.5, PostgreSQL 16, Redis 7, Docker Compose
Orchestration	stateful DAG engine - gates, parallel/join, approvals, retries, fallback, rollback, safe-stop, policies, hash-chained audit, lineage, metrics, dynamic re-planning
Scenarios	greenfield (bulk API) - brownfield (link expiry on real code) - ambiguous (safer, faster, who clicks)
Verification	68 automated tests; four recorded end-to-end runs; restart-recovery and Redis-outage checks
Generated	2026-11-01 by ReportBuilder.java from the repository documents and recorded runs

Contents

1. Final engineering summary
2. Architecture
3. The three scenarios
4. Testing approach, risks, trade-offs and limitations
5. Appendix - README (setup and usage)

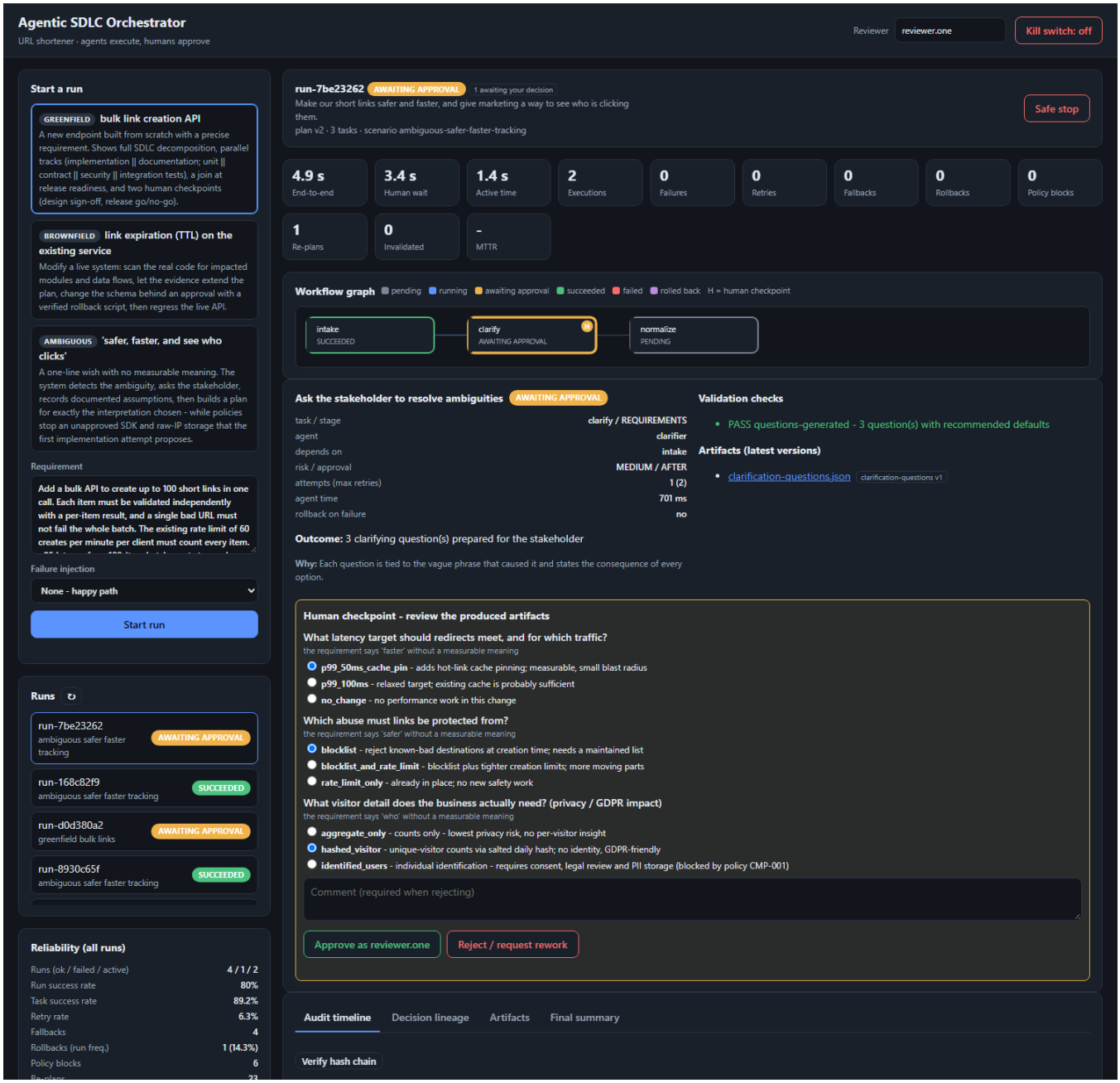
Final engineering summary

1. Outcome

A runnable prototype (`docker compose up -d --build`, dashboard on `http://localhost:8080/`) containing:

1. a production-style **URL shortener** (Spring Boot 3.5, Java 21, PostgreSQL, Redis), and
2. an **agentic SDLC orchestrator** that turns a requirement into reviewable engineering output - requirements, design, code, tests, documentation, release readiness - with a stateful DAG engine, entry/exit gates, parallel branches with joins, human approval checkpoints, bounded retries, fallback, rollback, safe-stop, policy guardrails, hash-chained audit, decision lineage, reliability metrics and dynamic re-planning.

Three scenarios (greenfield, brownfield, ambiguous) run end to end on the real stack; 68 automated tests pass; four recorded runs are committed under `docs/runs/`.



The dashboard at the human clarification checkpoint (ambiguous scenario)

2. Plan and rationale

The brief's differentiator is the orchestration layer, so the work was sequenced to protect it:

Step	What	Why this order
1	Shortener first (API, cache, rate limit, analytics, tests, Docker)	the orchestrator needs a real, changing codebase to reason about; it also makes the brownfield scenario honest
2	Engine core: model, state, gates, retries/fallback/rollback, approvals, safe-stop	the hardest-to-retrofit part; designed around one invariant - *only the engine changes state*
3	Policies + audit chain + metrics	governance must exist before agents do, so agents are born constrained
4	Agents that do **real** work (scan code, compile and run generated code, execute SQL, call the live API)	evidence beats assertions; deterministic so runs and tests are reproducible
5	Scenarios, REST API, dashboard, demo driver	make the behaviour visible and reviewable
6	Run everything, read the evidence critically, fix, document	six defects found this way (see Testing doc)

Key judgement calls (full table in ARCHITECTURE section 9): agents *propose*, the engine *disposes*; policies are code; re-execute only what genuinely changed; approvals are revoked when what was approved changes; release is a dry run.

3. Artifacts

Deliverable	Location
Working prototype	repository root; `docker compose up -d --build`; dashboard `/`; Swagger `/swagger-ui.html`
Architecture overview (components, orchestration model, control flow, decisions)	ARCHITECTURE.md
Three scenarios with decomposition, orchestration, validation	SCENARIOS.md + evidence in runs/
Setup instructions	README
Testing approach, limitations, trade-offs, risks	TESTING_AND_RISKS.md
API/schema definitions	Flyway migrations, springdoc OpenAPI for the shortener, generated OpenAPI + SQL + rollback per scenario in each run's artifacts
Unit/integration tests	`src/test` (68 tests)
PDF	Engineering-Report.pdf
Per-run final engineering summary	generated by the system: `GET /api/orchestrator/runs/{id}/summary`

4. Risks, trade-offs and validation - headline

- Highest risks: agents emitting unsafe output, agents escalating scope, rubber-stamp approvals, side effects after partial failure, schema changes. Each has an engine-level control and a test (risk register).
- Validation is by execution: generated Java is compiled and its checks run, migrations and rollbacks run on PostgreSQL in a scratch schema, regressions hit the live API, the contract is parsed and its references resolved, the audit chain is verifiable.
- Verified outside the test suite: restart mid-run, Redis outage, dashboard rendering.

5. Assumptions

Single trusted environment; reviewers are humans using the dashboard/CLI; PostgreSQL 16 + Redis 7 available; requirements are English text; scenarios are catalogued (new ones implement `ScenarioDefinition`).

6. Limitations

Agents are deterministic and rule-based (no LLM calls); approver identity is asserted, not authenticated; single-node scheduler; generated code runs in-process behind an API allow-list (not a sandbox); migration validation does not measure locking on large tables; no load test; Windows `mvnw.cmd` untested here. Details and mitigations in the testing document.

7. How AI assistance was used - and how it was kept honest

The system was built with Claude Code in a long, iterative session. The division of labour I would defend:

- **AI produced** first drafts of almost all code, tests, templates and documents.
- **Verification was not delegated.** Every behaviour claimed in these docs is backed by a failing-then-passing test or a recorded run. The first end-to-end run failed on its first task; the *system's own audit trail* identified the defect, which is the point of building audit-grade observability first. Later, reading the recorded evidence exposed a non-determinism bug that tests had not caught.
- **Human judgement stays where the brief puts it:** scope (what is real vs simulated), the autonomy boundary (what an agent may never do), what counts as high risk, and the decision to be explicit about limitations rather than hide them.
- The prototype embodies the same rule it enforces: *AI output is accepted only through gates that can say no.*

8. What I would do next

1. LLM-backed agents behind the existing contract, with the unsafe-code execution moved into an isolated container.
2. Real authentication/authorisation for approvers (OIDC), role-based approval rules per risk level.
3. Lease-based multi-instance scheduler (row locks), SSE instead of polling, external anchoring of audit hashes.
4. Testcontainers profile in CI, load test for the redirect path and the 500 ms bulk target, circuit breaker around Redis.
5. Policy-as-data (OPA/Rego) so security can change rules without a deploy.

9. Reviewing this in fifteen minutes

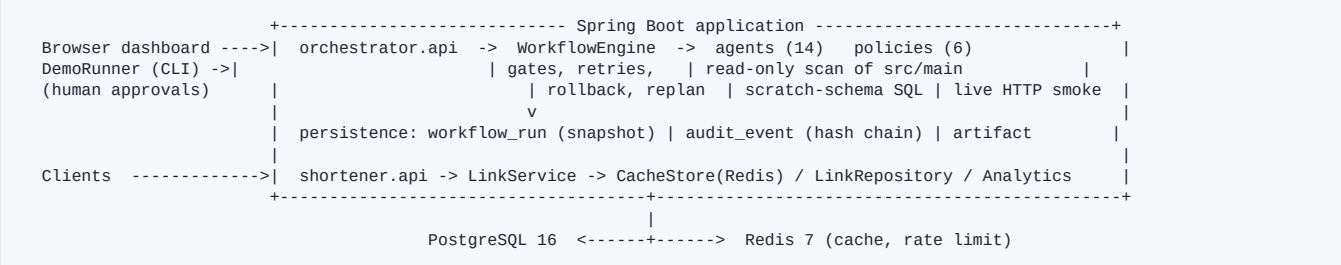
1. `docker compose up -d --build`, open `http://localhost:8080/`.
2. Start **Ambiguous** -> open node `clarify` -> answer `identified_users` for tracking -> approve. Watch the plan grow, the policy blocks (`impl-safety`, `impl-privacy`), the fallback, then approve design and the release.
3. Open the finished run's **Audit timeline** -> *Verify hash chain*; **Decision lineage**; **Final summary**.
4. Click `clarify` -> *Change an answer later* -> `aggregate_only` and watch dependants invalidate and approvals get revoked.
5. Start **Greenfield** with *Permanent failure* to see retries exhausted and rollback; **Brownfield** to see the plan grow from the code scan.
6. Read `engine/WorkflowEngine.java` (`advance`, `tryLaunch`, `onResult`, `handleFailure`, `failRun`) and `engine/PlanMutator.java`.

Architecture

1. Goals and design principles

1. **Controlled autonomy.** Agents do the work inside a sandboxed contract; the *engine* owns every state transition, gate, approval and plan change. An agent can only return artifacts, evidence and *proposals*.
2. **Governance is code.** Policies are evaluated by the engine at plan time, entry and exit. They are not prompts and an agent cannot negotiate with them.
3. **Everything is explainable.** Each decision (plan, replan, retry, fallback, approval, rollback, stop) is an audit event in a hash chain; each artifact is immutable, versioned and content-hashed; lineage links artifacts to the tasks that consumed them.
4. **Fail small, stay recoverable.** Bounded retries, fallbacks, reverse-order rollback, safe-stop. State survives a restart.
5. **Boring technology.** One deployable (modular monolith), PostgreSQL + Redis, no workflow framework - so the orchestration logic is visible, testable and reviewable (the whole `engine` package is about 1,300 lines).

2. System context



The orchestrator *governs changes to* the shortener: its agents read the shortener's source tree (read-only), validate migrations against the real database engine (scratch schema) and regress the running shortener over HTTP.

3. Components

Package	Responsibility
`orchestrator.model`	`TaskSpec` (immutable DAG node **and** governance envelope), `WorkflowRun`/`TaskState` (mutable state, JSON snapshot), `Artifact`, `AgentResult`, `ReplanRequest`, `Violation`
`orchestrator.engine`	`WorkflowEngine` (scheduler, gates, failure policy, approvals, safe-stop), `PlanMutator` (validated plan changes + invalidation), `DagValidator`, `WorkspaceService` (only place agents' output touches disk)
`orchestrator.policy`	`Policy` interface (plan/entry/exit checks), `PolicyEngine`, six policies
`orchestrator.agent`	`Agent` contract + 14 deterministic agents
`orchestrator.scenario`	`ScenarioDefinition` + three scenarios, `SdlcPlans` (shared verification/release sub-graph)
`orchestrator.persistence`	`RunRepository` (versioned snapshots, optimistic lock), `AuditLog` (hash chain), `ArtifactRepository`
`orchestrator.metrics`, `orchestrator.api`	reliability metrics; REST control plane; lineage and final-summary read models
`shortener.*`	the product: API, services, ports/adapters (`CacheStore`, `RateLimiter` with Redis and in-memory adapters)

4. Orchestration model

4.1 The plan is a DAG with a governance envelope

Each `TaskSpec` carries *what to run* and *how it may run*:

Field	Meaning
`dependsOn`	edges of the DAG. A task is runnable only when **all** dependencies are `SUCCEEDED` (join /synchronisation)
`risk` (LOW/MEDIUM/HIGH)	HIGH **must** have an approval checkpoint - rejected at plan time otherwise, also for tasks added mid-run
`approval` (NONE/BEFORE /AFTER)	BEFORE = a human authorises the action; AFTER = a human reviews the artifacts before dependants may use them
`requiredArtifacts`	exit-gate contract: the task fails if these artifact types are missing
`maxRetries`, `fallbackAgent`	bounded retry budget, then a more conservative agent
`rollbackOnFailure`	completed work is undone (agent hook + workspace cleanup) if the run later fails
`params`	agent inputs and policy flags (`schemaChange=true` demands a rollback script)

Branches without a dependency run **in parallel** (up to `max-parallel-tasks`, default 4, on virtual threads). The release readiness task is the synchronisation point: it starts only when every evidence track has finished.

4.2 Task lifecycle

```
PENDING --deps ok--> READY --entry gate--> RUNNING --exit gate ok--> SUCCEEDED
   ^                   |  (BEFORE approval)    |  \--exit gate fails--> retry (READY, backoff) --> fallback --> FAILED
   |                   v                       |  \--CRITICAL violation--> run SAFE_STOPPED (task back to PENDING)
   |             AWAITING_APPROVAL             v
   |                   | approve -> READY    AWAITING_APPROVAL (AFTER) --approve--> SUCCEEDED
   |                   | reject  -> FAILED          \--reject--> rework (PENDING, feedback) up to 2x --> FAILED
   +---- upstream output changed (invalidation): any started dependant is reset here, approvals revoked
FAILED/others on run failure: completed rollbackOnFailure tasks -> ROLLED_BACK, the rest -> CANCELLED
```

Run status: `RUNNING`, `AWAITING_APPROVAL` (nothing runnable, a human is needed), `SUCCEEDED`, `FAILED`, `SAFE_STOPPED`. If nothing is runnable and nothing awaits a human, the engine declares a *plan deadlock* and fails the run instead of hanging.

4.3 Scheduling and concurrency

- The engine is **event driven**: `advance(run)` is a scheduler tick, invoked on start, on every agent result, on approvals and on retry timers.
- All state transitions for a run happen under a **per-run lock** and are snapshotted to PostgreSQL (optimistic `version`). Agent execution runs **outside** the lock, so a slow agent never blocks approvals or other branches.
- Each `TaskState` has a **generation** counter. When upstream work is invalidated or a run is stopped, the generation is bumped and a late result from the old execution is discarded (`RESULT_DISCARDED`) rather than applied.
- On startup the engine re-queues tasks that were `RUNNING` when the process died (`TASK_RECOVERED`). Verified by restarting the container mid-run.

4.4 Gates

Entry gate (before an agent starts), in order: dependencies satisfied -> retry back-off elapsed -> parallelism cap -> `BEFORE` approval requested/recorded -> policy entry checks (approval integrity: recorded, human, not an `agent*` identity; release work needs every other task done) -> launch.

Exit gate (before anything the agent produced is accepted): policy scan of every artifact -> required artifacts present -> all agent-reported validation checks passed. Only then are artifacts persisted, context published and plan proposals applied.

Violation severity	Effect
`WARN`	recorded in the audit trail
`BLOCK`	exit gate fails; counts as a failed attempt (retry -> fallback -> fail). The violation text is fed back to the agent as feedback
`CRITICAL`	artifacts are **not persisted** (a leaked secret must not be copied into our own stores), the run is **safe-stopped**

4.5 Human approvals

- `decide(run, task, APPROVE|REJECT, approver, comment, answers)`; the approver identity is recorded in the audit trail.
- **Separation of duties:** identities starting with `agent` are refused. HIGH-risk approvals and every rejection need a written justification (>= 10 chars).
- `AFTER` rejection sends the task back for **rework** with the reviewer's text as feedback (max 2 reworks, then the run fails).
- `answers` carries structured human input (clarification answers) into the shared context.

4.6 Failure handling

Mechanism	Behaviour
Bounded retries	`maxRetries` per task, exponential back-off (`base * 2^(n-1)`, capped). Non-retryable agent errors (bad input, rejected plan proposal) go straight to failure - no retry, no fallback
Fallback	after retries are exhausted a `fallbackAgent` runs (fresh budget). Example: SDK-based reputation check blocked by policy -> in-house blocklist agent
Rollback	when a task finally fails, completed `rollbackOnFailure` tasks are unwound in **reverse completion order**: agent hook (e.g. drop scratch schema) + workspace cleanup. Best-effort; failures are audited as `ROLLBACK_FAILED` for manual clean-up
Safe-stop	no new task starts; in-flight results and human decisions are still recorded; only a human can `resume`
Kill switch	global emergency brake: safe-stops every active run and blocks `resume` until released
Autonomy budgets	max parallel tasks (4), max agent-proposed plan changes per run (5), max tasks per run (60), per-task timeout (60 s), artifact count/size caps. Exceeding a budget safe-stops

4.7 Dynamic re-planning (two distinct mechanisms)

1. **Plan proposals.** An agent returns a `ReplanRequest` (add tasks, prune pending ones, add dependencies). `PlanMutator` validates it - registered agents, policy `checkPlan` (e.g. HIGH risk needs approval), only not-yet-started tasks can be rewired, DAG stays acyclic, budgets hold - and records a `REPLAN` decision with the agent's rationale. Proposals are **idempotent**, so a re-executed agent can re-propose safely. *Examples:* the brownfield code scan adds `impl-cache-ttl` and `analytics-regression` only because it found those couplings in the real source; the ambiguous scenario grows from 1 to 14 tasks as answers arrive.
2. **Upstream-change invalidation.** When a task becomes `SUCCEEDED` its *committed hash* (artifact hashes, plus human answers for human-input tasks) is compared with the previous one. If it differs, every started dependant is reset to `PENDING`, **its approvals are revoked**, and in-flight results are orphaned. If the output is identical nothing downstream reruns. Triggered by a human `reopen` (e.g. a changed answer), by a rework, or by any re-executed task. Outputs must therefore be deterministic - a lesson learned when a test report embedding a random link code caused needless cascades (now fixed).

4.8 Context, lineage and audit

- **Cross-stage context:** a shared blackboard (`run.context`: ambiguity score, resolved capabilities, human answers, migration facts) plus the latest artifacts of each dependency, handed to the agent in `AgentContext`. Agents receive a deep copy.

- **Lineage:** every task records the ids of the artifacts it consumed (and an input hash). `/lineage` returns the artifact flow (producer -> consumer, version, hash) and the decision events that shaped the run.
- **Audit log:** append-only `audit_event`; each row stores the previous row's hash and `sha256(prev|run|seq|ts|actor|type|node|detail)`. Editing, deleting or reordering any event breaks verification (`/audit/verify`, covered by tests).
- **Metrics:** run/task success rate, retry rate, fallback and rollback frequency, MTTR (first failure -> eventual success per task), end-to-end latency and *active* latency (minus human wait), p50/p95, per scenario. Also exported through Micrometer (`/actuator/prometheus`).

5. Control flow of one task (sequence)

```
advance() tick  [lock]            agent thread (virtual)                      onResult()  [lock]
  deps met? approval? entry policies?
  mark RUNNING, attempts++, record consumed artifacts + input hash
  audit TASK_STARTED --------------> think / (chaos injection) / agent.execute(ctx)  [timeout 60 s]
                                                          |
                                                          +---------------> generation still current?  else discard
                                                                            exit gate (policies, required artifacts, checks)
                                                                              fail -> retry / fallback / failRun(rollback)
                                                                              CRITICAL -> safe-stop (nothing persisted)
                                                                            persist artifacts, publish context
                                                                            apply replans (validated) -> REPLAN event
                                                                            AFTER approval? AWAITING_APPROVAL : commitSuccess
                                                                            commitSuccess: MTTR sample, hash compare, invalidate
                                                                            save snapshot  -> advance()
```

6. Policy guardrails

Policy	Enforces	Phase	Severity
`SEC-001-secret-scan`	no credentials/keys/tokens in any artifact	exit	CRITICAL
`CMP-001-no-raw-pii`	schema/code must not store raw IP, e-mail, phone ... (hashes accepted)	exit	BLOCK
`SUP-001-dependency-allowlist`	only approved library groups may be introduced	exit	BLOCK
`SEC-002-unsafe-code`	generated source limited to a pure-computation JDK subset (no process/file/net /reflection/threads)	exit	BLOCK
`AUT-001-workspace-boundary`	artifact paths stay in the workspace; size and count caps	exit	BLOCK
`CHG-001-change-control`	HIGH risk needs a checkpoint; BEFORE needs a recorded human approval; schema change needs a rollback script; release needs all work done	plan / entry / exit	BLOCK

The security-review agent re-runs all exit policies over every upstream artifact as an independent second look.

7. Data model

Table	Notes
`workflow_run`	`id`, `status`, `version`, `state_json` (plan, task states, context, counters) - optimistic locking
`audit_event`	`(run_id, seq)` unique, `prev_hash`, `hash` - append-only chain
`artifact`	immutable, `(run, node, type, name, version)`, `sha256`, `content`
`short_link`, `click_event`	the product; `click_event` stores a salted daily visitor **hash**, never an IP

8. The shortener: key design points

- **Codes:** DB sequence -> multiplied by a constant coprime to 62^7 (a bijection) -> Base62. Unique **by construction** (no collision retry loop, verified over 200k ids), yet not sequentially guessable.
- **Redirect path:** cache-aside in Redis (`id|url`), **negative caching** (30 s) against cache-penetration, 302 (not 301) so deletions take effect, DB fallback if Redis is down.
- **Analytics is off the hot path:** bounded queue + worker pool; overflow is dropped and counted (`shortener.clicks.dropped`) - redirects never wait for analytics. Visitors are pseudonymised (SHA-256 of daily salt + address).
- **Rate limit:** fixed window per client address on creation only; **fails open** if Redis is unavailable.
- **Validation:** http/https only, no embedded credentials, no loopback/private targets, length cap; RFC 7807 errors that never leak internals.
- **Soft delete** keeps click history for audit.

9. Key decisions and trade-offs

Decision	Alternatives	Why / cost
Modular monolith, orchestrator in-process	separate service; Temporal / Camunda / Airflow	one artifact to run and review; the logic *is* the deliverable. Cost: no independent scaling
JSON snapshot per run + append-only audit	full event sourcing	simple, restart-safe, easy to query; cost: audit is evidence, not a rebuild source; snapshot rewrite per transition
Per-run lock + virtual threads	actors, reactive	straightforward reasoning about races; cost: single-node scheduler (see 11)
Agents **propose**, engine **disposes**	agents mutate the DAG	prevents a misbehaving agent from escalating its own scope
Policies as code in the engine	prompt-level guardrails	deterministic, testable, auditable
Committed-hash invalidation	always rerun / never rerun	re-executes exactly what changed; cost: needs deterministic outputs
Approvals BEFORE / AFTER	single mode	authorise risky *actions* vs review *artifacts* are different controls
Deterministic agents behind `Agent`	LLM calls	offline, reproducible demo and tests; LLM agent is a drop-in
H2 (PG mode) in tests	Testcontainers	works inside `docker build`/CI without Docker-in-Docker; mitigated by the migration validator and compose runs on real PostgreSQL
Dry-run release	real deploy	deployment needs credentials and environments outside scope; manifest pins what was reviewed

10. Security notes

- Approver identity is **asserted, not authenticated** in this prototype (production: SSO/OIDC). State-changing orchestrator calls can be protected with `X-API-Key`.
- Dashboard builds DOM with text nodes only (no `innerHTML` with server data).
- Agents write only to `workspace/{run}/{task}/`; reads of the source tree are read-only; migration validation uses a throw-away schema.
- Generated code is compiled and executed **in-process** by the unit-test agent behind an API allow-list policy. This is defence in depth, not a sandbox - untrusted LLM code should run in an isolated container (listed under limitations).

11. Scaling path (not built)

Multi-instance scheduling needs a lease per run (`SELECT ... FOR UPDATE SKIP LOCKED` or a distributed lock) instead of the in-process lock; events could be published through an outbox for the dashboard (SSE instead of polling); the agent pool would move to a queue with workers; Redis would hold the kill switch instead of process memory.

The three scenarios

Each scenario shows **decomposition** (how the requirement becomes a DAG), **orchestration** (what the engine did that a linear chain could not) and **validation** (evidence produced by actually running things). Every number below comes from a recorded run in `docs/runs/` (summary `.md` + full snapshot and audit trail `.json`).

Reproduce any of them:

```
docker compose up -d --build
docker compose --profile demo run --rm demo --scenario greenfield  --auto
docker compose --profile demo run --rm demo --scenario brownfield  --auto --failure transient
docker compose --profile demo run --rm demo --scenario ambiguous   --auto --answers tracking=identified_users --change tracking=aggrega
docker compose --profile demo run --rm demo --scenario greenfield  --auto --failure permanent      # rollback demonstration
```

---

1. Greenfield - bulk link creation API

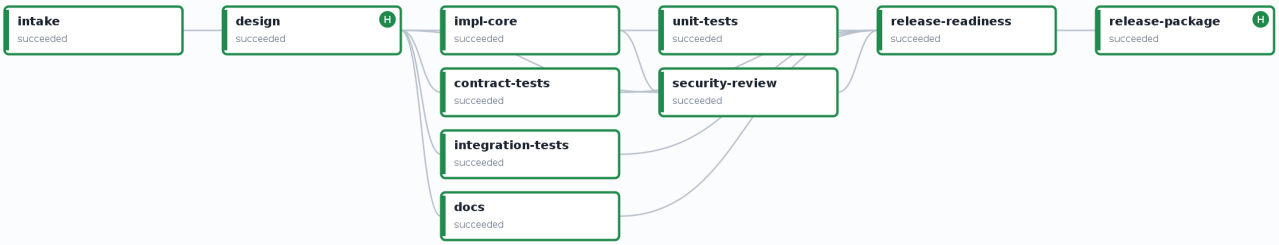
Requirement (well defined): *Add a bulk API to create up to 100 short links in one call. Each item validated independently with a per-item result; one bad URL must not fail the batch. The 60/min rate limit counts every item. p95 for 100 items < 500 ms.*

Decomposition

Requirement analysis finds measurable criteria (100 items, 60/min, 500 ms) -> ambiguity 0 -> no clarification needed, so the whole DAG is declared up front (10 tasks):

```
intake -> design [human review] -> impl-core ----+--> unit-tests ----------+
                      |                          +--> security-review -----+
                      +-> docs (parallel with impl) -------------------------+--> release-readiness -> release-package
                      +-> contract-tests ------------------------------------+      (join)            [human approval,
                      +-> integration-tests (live HTTP) --------------------+                          HIGH risk]
```

Greenfield - final graph of the recorded run (10 tasks, all succeeded)



dashed border = added by re-planning H = human checkpoint (green once approved) border colour = status (green succeeded, red failed, purple rolled back, grey cancelled)

Orchestration (what is non-linear)

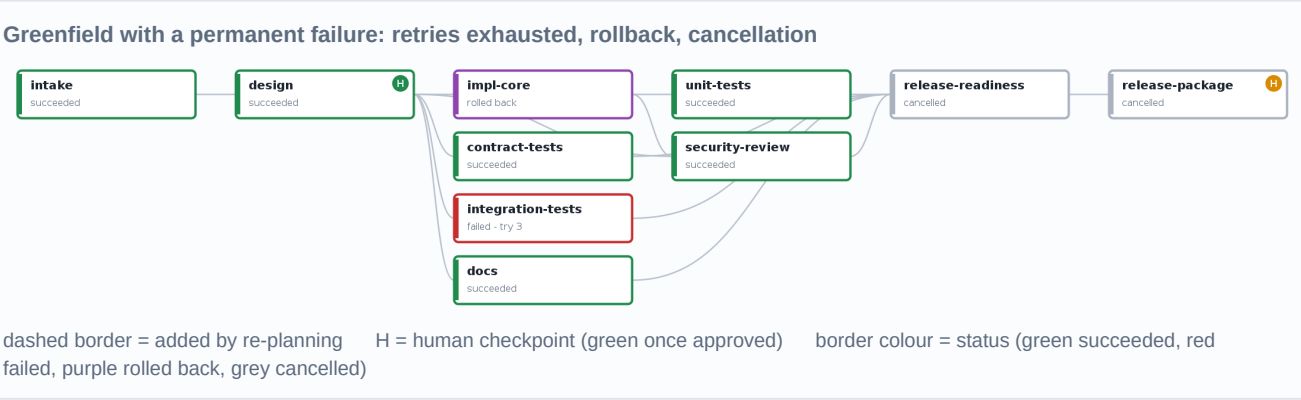
- After design sign-off, `impl-core`, `docs`, `contract-tests` and `integration-tests` start **in the same scheduler tick** (parallel fan-out); `unit-tests` and `security-review` follow `impl-core`.
- `release-readiness` is a **synchronisation join** over five evidence tracks - asserted in the test suite by event ordering.
- Two human checkpoints: design (`AFTER`, review artifacts) and release packaging (`BEFORE`, HIGH risk, justification required).
- Exit gates reject output unless required artifacts exist and every agent check passed.

Validation (real, not asserted)

Evidence	How it is produced

Generated `BulkLinkPlanner.java` + `BulkLinkPlannerCheck.java`	compiled in memory with the JDK compiler and the checks **executed** (batch limits, partial failure, de-duplication)
OpenAPI contract	parsed; versions, operations, responses, every `$ref` resolved
Baseline regression	live HTTP against the running service: create 201, redirect 302, metadata, stats, unsafe URL 400, delete then 404
Release readiness	verdict recomputed from the evidence artifacts; evidence hashes pinned in `release-bundle.json` (dry run)

Recorded run `greenfield-none-run-861164ef`: SUCCEEDED, 10 tasks, 10 executions, 0 retries, 27 audit events, 5.4 s end to end (0.5 s human wait), audit chain intact.



Failure injection on the same scenario

- `--failure transient`: `unit-tests` and `integration-tests` each crash once -> 2 bounded retries (exponential back-off) -> recovered; **MTTR** is recorded per task.
- `--failure permanent`: `integration-tests` fails every attempt. Recorded run `greenfield-permanent-run-dfa8a339`: 1 attempt + 2 retries, then the engine fails the run, **rolls back** `impl-core` (agent hook + workspace deleted), cancels `release-readiness` and `release-package`. Counters: 3 failures, 2 retries, 1 rollback.

---

2. Brownfield - link expiration on the existing service

Requirement: *Links support an optional expiry. After expiry the redirect returns 410 Gone, cached entries must not outlive the expiry, existing click history is preserved, existing links keep working.*

Codebase reasoning (on the real source tree)

`codebase-analysis` scans `src/main` (read-only) with probes for the symbols this change must touch, and produces an **impact report**: impacted files with line-level evidence, modules, API impact, schema impact and data flows, e.g.

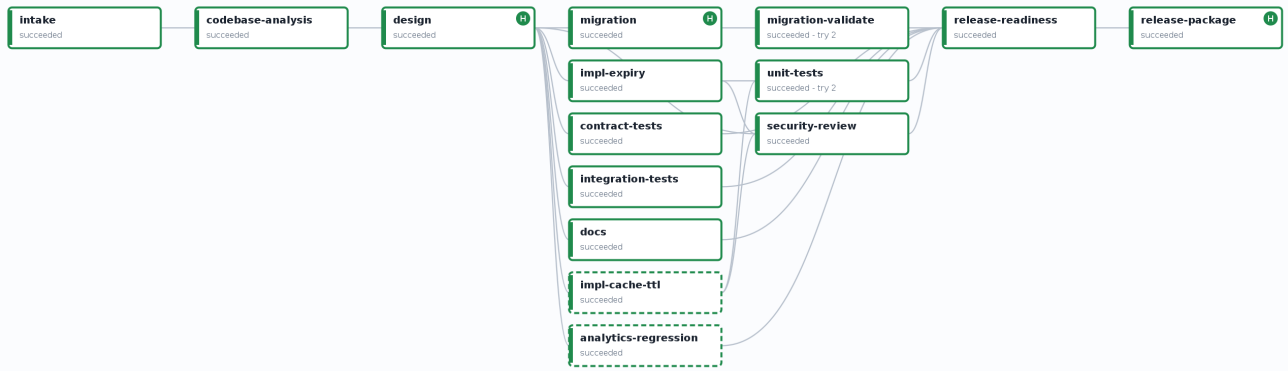
- `shortener/domain/ShortLink.java` (entity gains `expiresAt`), `infra/LinkRepository.java` + `db/migration/V1__*` (`short_link`), `service/LinkService.java` (lookup path **and** `CacheStore` coupling), `api/RedirectController.java` (new 410 outcome), `domain/LinkException.java` (error kind), `api/LinkController.java` (stats shares the lookup).
- Data flow: `GET /{code} -> RedirectController -> LinkService.resolve -> CacheStore(link:<code>) -> LinkRepository.findByCode -> short_link`
    - *expiry must be enforced on both the cache hit and the DB read.*

Decomposition - the plan grows from evidence

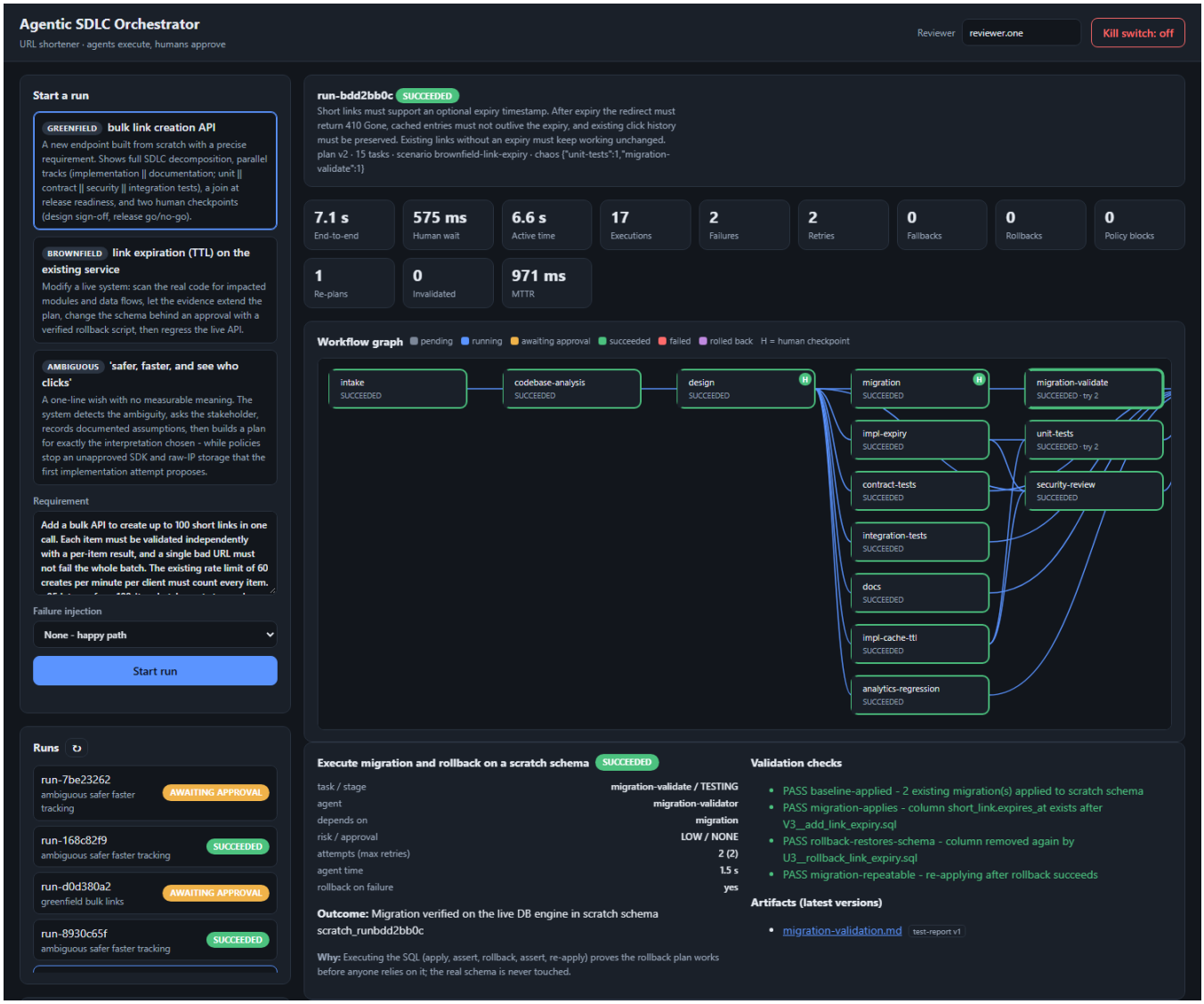
Initial DAG: 13 tasks. The scan found cache coupling and stats/lookup coupling, so the analysis agent proposed two more tasks, which the engine validated and added (`REPLAN`, plan v2 -> **15 tasks**): `impl-cache-ttl` (cache TTL capped by expiry) and `analytics-regression`, and rewired `unit-tests`, `security-review` and `release-readiness` to wait for them.

```
intake -> codebase-analysis --(evidence)--> design [human review]
design -> migration [HIGH, human review] -> migration-validate -----------+
design -> impl-expiry, impl-cache-ttl* -> unit-tests, security-review ----+
design -> contract-tests, integration-tests, docs, analytics-regression* -+--> release-readiness --> release-package
                                                                              (join)           [HIGH, approval]
                                                  (* added by the re-plan from code-scan evidence)
```

Brownfield - 13 planned tasks, 2 added by the re-plan (dashed)



dashed border = added by re-planning H = human checkpoint (green once approved) border colour = status (green succeeded, red failed, purple rolled back, grey cancelled)



Dashboard: brownfield run, migration validation evidence

Orchestration and governance

- Schema change is **HIGH risk** -> mandatory human checkpoint; the change-control policy also refuses a schema task that does not ship a **rollback script**.
- The migration and code tracks run in parallel and join at release readiness.
- **Change control in action:** `migration-validate` executes V1 + V2 + the candidate migration on the real PostgreSQL engine in a throw-away schema, asserts the new column, runs the **rollback script**, asserts the column is gone, re-applies, then drops the schema. (`migration-applies`, `rollback-restores-schema`, `migration-repeatable` all PASS.)
- Transient failure injection: `migration-validate` and `unit-tests` each fail once and recover (MTTR 971 ms).

Recorded run `brownfield-transient-run-bdd2bb0c`: SUCCEEDED, 15 tasks, 17 executions, 2 retries, 1 re-plan, 46 audit events, 7.1 s end to end (0.6 s human wait).

---

3. Ambiguous - "safer, faster, and see who clicks"

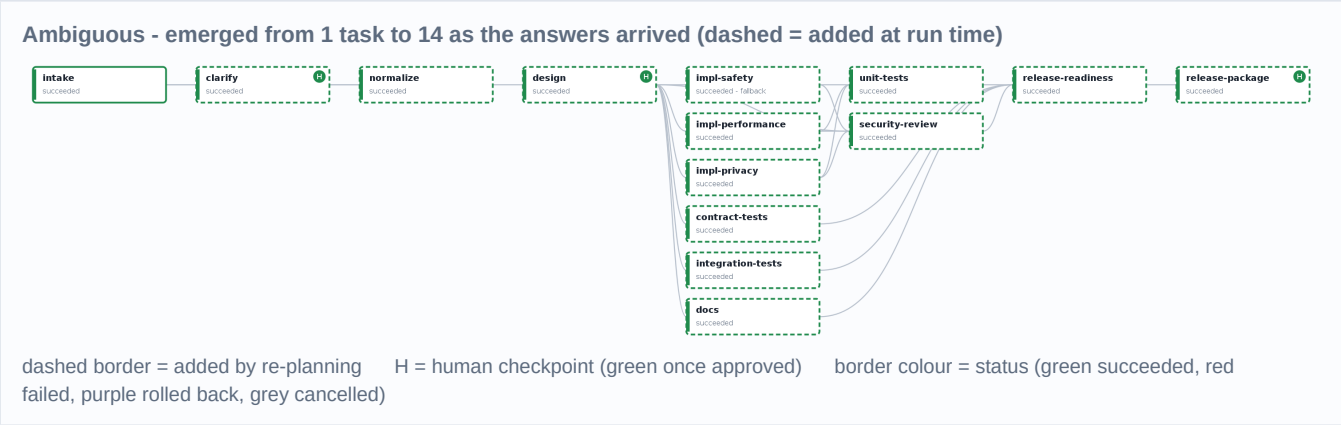
Requirement: *Make our short links safer and faster, and give marketing a way to see who is clicking them.*

Understanding

The analyst scores ambiguity **1.0**: three vague terms (`faster`, `safer`, `who`) and no measurable criteria. It does **not** plan on guesses. Only `intake` exists up front (1 task).

Decomposition emerges at run time (1 -> 3 -> 14 tasks)

1. `intake` proposes `clarify` + `normalize` (re-plan v2): *ambiguity >= 50% -> insert a human clarification checkpoint*.
2. `clarify` produces three multiple-choice questions, each with the consequence of every option and a recommended default. The stakeholder answers at the approval checkpoint (dashboard form or CLI).
3. `normalize` folds the answers into explicit capabilities and **assumptions**, then proposes the rest of the SDLC graph - only the tracks the answers justify (re-plan v3): design, `impl-safety`, `impl-performance`, `impl-privacy`, verification tracks, release.



Guardrails stop the agents' first attempts (real policy decisions, not staged failures)

The stakeholder chose the risky option `identified_users`:

Event	What happened
`impl-safety`	primary `codegen` proposed a third-party reputation SDK -> `SUP-001` **blocked** (`com.shady:phishguard-sdk` not on the allow-list) -> retries 0 -> **fallback** `codegen-conservative` produced an in-house host blocklist
`impl-privacy`	first attempt stored `ip_address` -> `CMP-001` **blocked** -> **retry with the policy message as feedback** -> hashed visitor schema (`visitor_hash`), and the rationale records that individual identification cannot be built under policy
Unsafe artifacts	never persisted (asserted: no `phishguard`, no `ip_address VARCHAR` anywhere in the artifact store)

Upstream change -> dynamic re-plan with governance preserved

After completion the stakeholder changes the tracking answer to `aggregate_only` (`--change tracking=aggregate_only`, dashboard "Change an answer later"). `clarify` is re-opened; its committed hash changes, so the engine **invalidates only dependants whose output changes**, hop by hop (`normalize` -> `design` -> impl tracks -> unit/security -> readiness -> release):

- 42 task resets, **approvals revoked and given again** (design and release were approved twice).
- `contract-tests`, `integration-tests`, `docs` re-ran but produced identical output, so nothing below them cascaded needlessly.
- `impl-privacy` now generates the aggregate-only counter and a `daily_click` schema.

Recorded run `ambiguous-none-run-168c82f9-change`: SUCCEEDED, 14 tasks (plan v3), 30 executions, 1 retry, **2 fallbacks**, **3 policy blocks**, 95 audit events, 15.0 s end to end (2.0 s human wait), audit chain intact.

Recorded runs at a glance

Run (evidence in docs `/runs/`)	Result	Tasks (plan v)	Executions	Retries	Fallbacks	Rollbacks	Policy blocks	Re-plans	Invalidated	MTTR	E e (h w
greenfield, no failures (`861164ef`)	SUCCEEDED	10 (v1)	10	0	0	0	0	0	0	-	5 ((
greenfield, permanent failure (`dfa8a339`)	FAILED	10 (v1)	10	2	0	1	0	0	0	-	4 ((
brownfield, transient failures (`bdd2bb0c`)	SUCCEEDED	15 (v2)	17	2	0	0	0	1	0	971 ms	7 ((
ambiguous + answer change (`168c82f9`)	SUCCEEDED	14 (v3)	30	1	2	0	3	10	42	781 ms	1 (2

Re-plans counts every re-planning event (agent proposals, human re-opens, upstream-change invalidations); *Invalidated* counts task resets caused by upstream changes. Latencies include the artificial agent "think time" (700 ms per task) that makes progress visible in the dashboard.

Testing approach, risks, trade-offs and limitations

1. Testing approach

68 automated tests, all green (`./mvnw test` or `docker compose --profile test run --rm tests`), plus manual verification on the real PostgreSQL/Redis stack. The strategy is *test the behaviour that makes the system trustworthy*, not just the happy path.

Suite	Tests	What it proves
`CodeGeneratorTest`	4	codes are unique over 200k ids, fixed length/alphabet, non-sequential, reject out-of-space ids
`UrlValidatorTest`	21	scheme/credential/private-host/length rules, incl. boundary cases (`172.32.x` is public, `172.20.x` private)
`ShortenerApiIntegrationTest`	7	create -> redirect -> stats (async clicks drained), delete -> 404, negative cache, rate limit 429 + redirects unthrottled, RFC 7807
`DagValidatorTest`	4	diamond accepted; cycles, self/unknown dependencies, duplicates rejected; transitive downstream order
`PolicyTest`	8	every policy: positive and negative cases, severities, entry-gate approval integrity, release entry gate
`ScenarioIntegrationTest`	8	the three scenarios end to end (below)
`GovernanceIntegrationTest`	9	approval rules, rework and rework budget, safe-stop/resume, kill switch, secret leak, agent over-reach, runaway re-planning
`OrchestratorApiTest`	3	REST lifecycle over real HTTP, error mapping (404/400/409), metrics shape
`AuditLogIntegrationTest`	2	chain links; editing or deleting an event breaks verification
`LlmAdvisorTest` + advisor stand-in in the scenario tests	2	advisor is off without a key; a stand-in that returns different text on every call never triggers re-planning (advisory artifacts are excluded from the output hash)

`ScenarioIntegrationTest` asserts orchestration properties, not just "status = SUCCEEDED": parallel fan-out by event ordering, the join (readiness starts after all five evidence tasks), approvals recorded with identity, retry counts and MTTR samples, rollback (task `ROLLED_BACK`, workspace folder deleted, dependants `CANCELLED`), plan growth from real code-scan evidence, policy blocks, fallback, that blocked artifacts were never persisted, and that changing an answer invalidates dependants and revokes approvals - while an unchanged re-run does **not** cascade.

Why integration tests carry the weight. The risk in this system is in the interactions (gates x retries x approvals x re-planning), so those tests boot the real Spring context, call the live HTTP API (the smoke agent hits the test server), compile and execute generated code, and run migrations on a real SQL engine.

Verified beyond the test suite (real stack, `docker compose`)

- Four recorded runs on PostgreSQL 16 + Redis 7 (`docs/runs/`), including the migration + rollback executed in a PostgreSQL scratch schema.
- **Restart recovery:** the app container was restarted mid-run; orphaned `RUNNING` tasks were re-queued (`TASK_RECOVERED`), the run completed to the release checkpoint and the audit chain still verified.
- **Redis outage:** with Redis stopped, create and redirect still succeed (database fallback), liveness stays UP; after Redis returns the redirect is served in ~35 ms again.
- **Dashboard** rendered in a headless browser (screenshots in `docs/images/`), JavaScript syntax-checked.
- Maven wrapper bootstraps Maven 3.9.9 inside a JDK-only container.

Defects found by this process (and fixed) - evidence the validation is real

1. `MAX(version)` over zero rows returned NULL and `JdbcClient.single()` refused it - **found because the audit trail of a failed run pointed straight at it** (first end-to-end run failed at task 1).

2. The smoke agent's base URL used a `${local.server.port}` placeholder that cannot be resolved at bind time; the agent now reads the actual port lazily.
3. Re-plan budget counted every cascade hop, so a legitimate upstream change safe-stopped the run; the budget now bounds only *agent-proposed* plan changes, cascades are bounded by the DAG.
4. When the re-plan budget tripped, the triggering task stayed `RUNNING` forever (run never settled); it now returns to `PENDING`.
5. The integration-test report embedded a random link code, so every re-run looked like a changed output and revoked release approvals. Found while reading the recorded evidence; outputs must be deterministic - documented in the architecture.
6. "Human wait" ignored approvals still pending, so a parked run looked like slow agent work.

2. Risk register

#	Risk / failure scenario	Likelihood	Impact	Controls in the system	Residual
1	Requirement misunderstood, work built on a guess	M	H	ambiguity scoring, clarification checkpoint, assumptions recorded, human sign-off on design	analyst is rule-based; subtle ambiguity may pass
2	Agent emits unsafe artifact (secret, PII, unapproved library, dangerous API)	M	H	six exit policies, CRITICAL -> safe-stop and **no persistence**, independent security re-scan	pattern-based detection can be evaded
3	Agent escalates its own scope (adds risky tasks, endless replans)	M	H	proposals validated by the engine; HIGH risk needs a checkpoint; plan-change and task budgets; safe-stop	budgets are static
4	Rubber-stamp approvals	M	H	identity recorded, separation of duties, written justification for HIGH risk and rejections, revocation on upstream change	identity is asserted, not authenticated
5	Half-finished change leaves side effects	M	M	reverse-order rollback of side-effecting tasks, workspace cleanup, `ROLLBACK_FAILED` surfaced	rollback is best-effort
6	Schema migration breaks production	L	H	additive/nullable design, rollback script mandatory, migration + rollback **executed** on real engine, HIGH-risk approval	not tested on production-sized data/locks
7	Expired/deleted link still served from cache	M	M	TTL capped by expiry, eviction on delete, 302 not 301, short negative cache	seconds of skew across nodes
8	Audit trail tampered with	L	H	hash chain + verify endpoint	an attacker who can rewrite the whole chain defeats it; anchor hashes externally in production
9	Non-deterministic agent output causes invalidation storms	M	M	committed-hash comparison; deterministic reports; tests for "unchanged -> no cascade"	LLM agents need normalisation of volatile text
10	Orchestrator crash mid-run	M	M	snapshot per transition, generation counters, `TASK_RECOVERED` on startup	duplicate execution of a side-effecting agent is possible (agents are idempotent here)
11	Redis outage	M	M	cache and rate limiter fail open/fall back to DB; verified manually	each request pays the 200 ms timeout while Redis is down - add a circuit breaker
12	Analytics loss under load	M	L	bounded queue, drops counted and exported	analytics is best-effort by design

13	Abuse of link creation / phishing destinations	M	M	rate limit per client, URL validation, blocklist capability (ambiguous scenario)	fixed window allows 2x burst at boundaries
14	Generated code executed in-process	L (trusted templates)	H (untrusted LLM)	restricted-API policy, platform-only classloader	**not a sandbox**; isolate in a container before using an LLM agent
15	Single-node scheduler is a SPOF	M	M	state is durable; restart recovers	needs lease-based scheduling to scale out

3. Trade-offs made (and why)

- **Deterministic agents over LLM calls** - reproducible demo and tests, zero credentials; the `Agent` contract is the seam for an LLM.
- **Snapshot + audit chain over event sourcing** - far less machinery; audit is evidence, not the source of truth.
- **Modular monolith over a workflow product** - the orchestration logic is the thing being evaluated, so it is plain code.
- **H2 in tests over Testcontainers** - runs inside Docker builds/CI; the real-engine gap is covered by the migration validator and compose runs.
- **Hash-based minimal re-execution over "always rerun"** - cheaper and preserves approvals when nothing changed, at the cost of demanding deterministic outputs.
- **Fail-open rate limiter** - availability of link creation beats strict throttling during a cache outage.
- **Release as dry run** - deployment adds credentials and environments that are out of scope; the pinned manifest is the contract with a real pipeline.

4. Limitations

- Agents do not call an LLM; "requirement understanding" is rule-based (lexicon + measurability heuristics).
- Approver identity is not authenticated; the optional API key is a stopgap, not authN/authZ.
- Scale-out of the orchestrator (multiple instances) is not implemented; kill switch lives in process memory.
- The dashboard polls (1.2 s) instead of using server-sent events.
- The migration validator proves apply/rollback semantics, not lock behaviour or runtime on large tables.
- No load/performance test: the 500 ms bulk target is a requirement the scenario *carries*, not one this prototype measures.
- `mvnw.cmd` (Windows wrapper) was not run here; Docker and the Linux wrapper were.
- Chaos injection simulates agent crashes; it does not model slow agents beyond the per-task timeout.

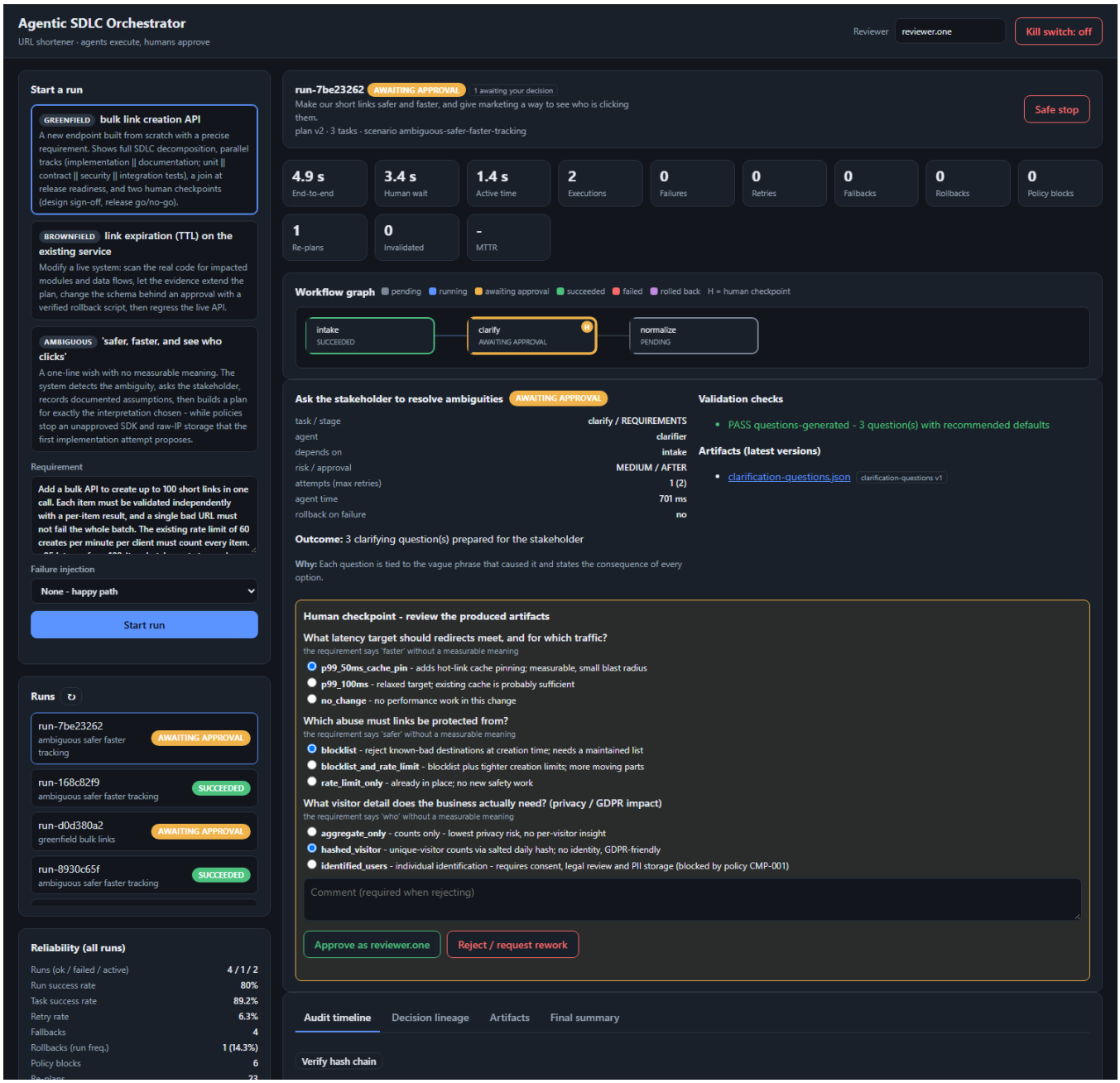
5. Assumptions

- A single trusted operator environment (no multi-tenant isolation); reviewers use the dashboard or CLI.
- PostgreSQL 16 and Redis 7 are available (compose provides both); JDK 21 only for local development.
- Requirements are English text; scenario catalogue is fixed (new scenarios implement `ScenarioDefinition`).
- "Production-grade" is interpreted as: clear boundaries, tests, safe failure behaviour and observability - not hardened infrastructure.

Agentic SDLC Orchestrator - URL Shortener

A URL shortener (Java 21 / Spring Boot / PostgreSQL / Redis) **plus a governed agentic orchestration layer** that turns a requirement into a reviewable engineering outcome: requirement analysis, design, implementation, tests, documentation and release readiness - executed by agents under explicit autonomy boundaries, with humans owning approvals and final quality.

> **Agents execute. Humans approve. Policies cannot be waived by agents.** Nothing in this prototype deploys anything: the release step assembles a hash-pinned *dry-run* bundle.



What is in the box

Part	What it is

Shortener service	`POST/GET/DELETE /api/v1/links`, `GET /{code}` (302), `GET /api/v1/links/{code}/stats`. Collision-free codes, Redis cache-aside with negative caching, fixed-window rate limit, URL safety validation, async click analytics with privacy-preserving visitor hashes, RFC 7807 errors, Flyway, Micrometer/Prometheus.
Orchestrator	Stateful DAG engine: entry/exit gates, parallel branches + joins, human approval checkpoints, bounded retries, fallback agents, rollback, safe-stop + global kill switch, dynamic re-planning, hash-chained audit log, decision lineage, reliability metrics (success rate, retry/rollback rate, MTTR, latency).
Policies	Secret scan, no-raw-PII, dependency allow-list, unsafe-code, workspace boundary, change control (approval + rollback script). Code, not prompts - agents cannot waive them.
14 agents	Requirements analyst, clarifier, architect, codebase analyst, migration author/validator, codegen (+ conservative fallback), unit-test runner, contract tester, smoke tester, security reviewer, doc writer, release manager/packager. Deterministic and offline; an LLM-backed agent implements the same `Agent` contract.
Dashboard	`http://localhost:8080/` - live DAG, approvals, answers to clarification questions, audit timeline with hash-chain verification, decision lineage, artifacts, auto-generated final summary.
3 scenarios	Greenfield (bulk API), Brownfield (link expiry on the real code), Ambiguous ("safer, faster, see who clicks") - see docs/SCENARIOS.md.

Quick start (Docker only - no local Java needed)

```
docker compose up -d --build          # postgres + redis + app
# open the dashboard:   http://localhost:8080/
# API docs (Swagger):   http://localhost:8080/swagger-ui.html
```

Run the three scenarios from the command line with the Java driver (`DemoRunner`; Docker needs nothing else installed):

```
docker compose --profile demo run --rm demo --auto                                  # all three, approvals automatic
docker compose --profile demo run --rm demo --scenario brownfield --failure transient --auto   # retries + MTTR
docker compose --profile demo run --rm demo --scenario greenfield --failure permanent --auto   # retries exhausted -> rollback
docker compose --profile demo run --rm demo --scenario ambiguous --auto --answers tracking=identified_users --change tracking=aggregate

# same driver from a local JDK (interactive: omit --auto and approve every checkpoint in the terminal)
./mvnw -q compile exec:java -Dexec.args="--scenario ambiguous"
```

Or use the dashboard: pick a scenario, press **Start run**, click nodes of the graph, answer the checkpoints. Every run writes its final engineering summary to `docs/runs/`.

Local development (JDK 21)

```
docker compose up -d postgres redis
./mvnw spring-boot:run                # Windows: mvnw.cmd spring-boot:run
```

The Maven wrapper bootstraps Maven 3.9.9 (verified on Linux; `mvnw.cmd` is the Windows equivalent). The app reads `DB_URL`, `DB_USER`, `DB_PASSWORD`, `REDIS_HOST`, `REDIS_PORT` (defaults point at `localhost`).

Tests

```
./mvnw test                                        # JDK + Maven locally
docker compose --profile test run --rm tests       # no local JDK: runs inside a Maven container
```

68 tests: unit, policy, DAG, and full-stack integration tests (real Spring context, live HTTP, H2 in PostgreSQL mode). See docs/TESTING_AND_RISKS.md.

Optional: real Claude in the loop

Set `ANTHROPIC_API_KEY` and the requirements analyst asks Claude (`claude-opus-5-5`, override with `ANTHROPIC_MODEL`) for a second opinion on every requirement: ambiguities, missing acceptance criteria, risks, questions. It appears as the artifact `llm-review.md`.

```
ANTHROPIC_API_KEY=sk-ant-... docker compose up -d --build       # PowerShell: $env:ANTHROPIC_API_KEY="..."; docker compose up -d --bui
```

Design rules: the advice is **advisory** (separate `advisory-*` artifact, excluded from the output hash, never drives planning or gates), so with no key - or if the call fails - everything behaves exactly as before. The key is read from the environment only; it is never logged, stored or written into artifacts. Status: unit/integration tested with a stand-in advisor (random text on every call, proving it cannot trigger re-planning); **not yet exercised against the live API here, because no key was available**.

Try the shortener itself

In the browser: **http://localhost:8080/ui/shortener.html** (create, open, stats, delete). The orchestrator dashboard is at `http://localhost:8080/` and the two pages link to each other. Or use the API directly:

```
curl -s -X POST localhost:8080/api/v1/links -H 'Content-Type: application/json' -d '{"url":"https://example.com/hello"}'
# {"code":"Nhcfe13","shortUrl":"http://localhost:8080/Nhcfe13",...}   (your code will differ)
curl -i localhost:8080/Nhcfe13                        # 302 Location: https://example.com/hello
curl -s localhost:8080/api/v1/links/Nhcfe13/stats     # clicks, unique visitors (hashed), referrers
```

Orchestrator API (what the dashboard uses)

Method & path	Purpose
`GET /api/orchestrator/scenarios`	scenarios, default requirements, plan rationale
`POST /api/orchestrator/runs` `{scenario, requirement?, failureMode, actor}`	start a run
`GET /api/orchestrator/runs/{id}`	plan (DAG), task states, context, metrics
`POST .../runs/{id}/tasks/{task}/decision`	approve / reject a checkpoint (`answers` for clarification)
`POST .../runs/{id}/tasks/{task}/reopen`	change an upstream input -> dependants re-plan
`POST .../runs/{id}/safe-stop`, `/resume`, `POST /kill-switch`	emergency controls
`GET .../runs/{id}/audit`, `/audit/verify`	audit trail, hash-chain verification
`GET .../runs/{id}/lineage`, `/artifacts`, `/summary`	decision lineage, artifacts, final engineering summary
`GET /api/orchestrator/metrics`	success rate, retry/rollback frequency, MTTR, p50 /p95 latency

Set `ORCHESTRATOR_API_KEY` to require `X-API-Key` on all state-changing orchestrator calls.

Project layout

```
src/main/java/com/sa/agentic
  shortener/        domain | api | service | infra (ports: CacheStore, RateLimiter; adapters: Redis / in-memory)
  orchestrator/
    model/          TaskSpec (DAG node + governance envelope), WorkflowRun, TaskState, Artifact, AgentResult ...
    engine/         WorkflowEngine (scheduler, gates, retry/fallback/rollback, safe-stop), PlanMutator, DagValidator
    policy/         Policy + PolicyEngine + 6 policies
    agent/          Agent contract + 14 shipped agents
    scenario/       ScenarioDefinition + 3 scenarios + reusable SDLC sub-graphs
    persistence/    RunRepository (versioned snapshots), AuditLog (hash chain), ArtifactRepository
    metrics/        MetricsService          api/  REST controller, insights (lineage, final summary)
src/main/resources  db/migration (Flyway) | static/ui (dashboard)
src/test            shortener + orchestrator tests         demo/DemoRunner   scenario driver (CLI)
docs/               ARCHITECTURE | SCENARIOS | TESTING_AND_RISKS | FINAL_ENGINEERING_SUMMARY | runs/ (evidence) | Engineering-Report.pd
```

Assignment map

Requirement	Where
Requirement understanding, ambiguity	`RequirementsAnalystAgent`, `ClarifierAgent`, ambiguous scenario
Task decomposition with dependencies	`TaskSpec` DAG, `SdlcPlans`, scenarios; graph in dashboard
Brownfield codebase reasoning	`CodebaseAnalystAgent` scans the real source tree; plan grows from its evidence
Orchestration: gates, parallel/sync, context + lineage, approvals, retries/fallback/rollback/safe-stop, policies, audit, metrics, re-plan	`engine/`, `policy/`, `persistence/AuditLog`, `metrics/` - ARCHITECTURE
Production-quality output: code, API/schema, tests, docs	shortener + generated artifacts (OpenAPI, SQL migration + rollback, Java + executable checks, API/runbook/changelog)
Validation and risk control	TESTING_AND_RISKS; real compile-and-run, scratch-schema migration, live HTTP regression
Controlled autonomy	approval modes BEFORE/AFTER, separation of duties, autonomy budgets, workspace boundary
Final engineering summary	FINAL_ENGINEERING_SUMMARY + generated per run (`/summary`)
Three scenarios	SCENARIOS, evidence in `docs/runs/`
PDF	docs/Engineering-Report.pdf

Honest scope notes

- Agents are **deterministic and offline** (a deliberate choice for a reproducible demo; the brief does not say either way); they do real work (scan code, compile and run generated code, execute SQL, call the live API) but do not call an LLM by default (an optional Claude advisory review exists, see above). Swapping in a fully LLM-driven agent changes one bean.
- The release step is a **dry run**; deployment is out of scope on purpose.
- Limitations and trade-offs are listed in docs/TESTING_AND_RISKS.md.

Everything is Java, including the tooling. The PDF is regenerated from the docs and the recorded runs with `./mvnw -q test-compile exec:java@report` (`src/test/.../report/ReportBuilder.java`: commonmark + Java2D graphs + OpenHTMLtoPDF; test-scope dependencies only, nothing extra ships in the application jar). On a minimal Linux JDK image install `fontconfig fonts-dejavu` first.

Built with AI assistance (Claude Code). Every claim above is backed by a test or a recorded run; what AI produced and how it was verified is described in the final summary.